



Guião 4 – GUI TKinter

Objectivos:

- Desenvolvimento de uma aplicação gráfica simples para apresentação e edição dos dados de uma pessoa.

1. Vamos começar pelo modelo dos dados. Num ficheiro python desenvolva uma classe que modele os dados de uma pessoa. A classe deverá apresentar informação como nome (primeiro e último), ano de nascimento, morada, etc.

Atente ao tipo de dados que pretende utilizar. P.ex: nome:str, a data de nascimento:datetime, a altura:float.

Adicione também código de teste nesse ficheiro, nomeadamente, instanciar objetos e fazer o print dos seus campos, via `__str__`.

2. Num ficheiro separado desenvolva o código que permitirá apresentar os vários campos numa janela tkinter.

Coloque esse código dentro de uma função (ou numa classe) e assuma que um dos argumentos da função será um objeto da classe descrita no modelo anterior.

3. Ainda no ponto anterior, inclua um botão que permita ativar um modo de edição. Assim, a janela deverá apresentar um botão “Editar” que comuta a vista para modo de edição, trocando as widgets por campos de edição.

As widgets deverão ter a capacidade para se auto-validarem. Ou seja, sempre que o utilizador submeter um campo, deverá ser executada uma função de validação e que indique ao utilizador um alerta se o campo estiver errado. P.exemplo, o formato da data ou a altura que esteja fora de parâmetros convencionais para uma pessoa.

Analise a possibilidade de criar uma classe (subclasse de Frame) que agreguem várias widgets e que sejam autónomas nesse sentido.



4. Estando em modo de edição, deverá surgir um botão “Guardar” que permita recolher toda a informação, validar a falta de campos obrigatórios e se OK, criar um documento no formato json.

5. Finalize o guião e envie via moodle.

GUI TKinter

1. Classe Pessoa com reescrita do método `__str__` e código de teste:

```
Semana9 > ex1.py > Pessoa > __str__
1  #from datetime import datetime
2  from datetime import date
3  from dataclasses import dataclass
4
5  class Pessoa:
6
7      def __init__(self, primeiro_nome : str , ultimo_nome : str, data_nascimento : date , morada : str, altura : float):
8          self.first_name = primeiro_nome
9          self.last_name = ultimo_nome
10         self.birthdate = data_nascimento
11         self.address = morada
12         self.height = altura
13
14     def __str__(self):
15         return f"Primeiro Nome: {self.first_name}\nÚltimo Nome: {self.last_name} \nData de Nascimento: {self.birthdate}\nMorada: {self.address}\nAltura: {self.height}\n"
16
24
25  #código de teste do ex1.py:
26  if __name__ == "__main__":
27
28      pessoa_1 = Pessoa("Rui", "Oliveira", date(2000, 5, 17), "Rua 25 abril", 1.90)
29      print(pessoa_1)
```

Para testar a classe Pessoa:

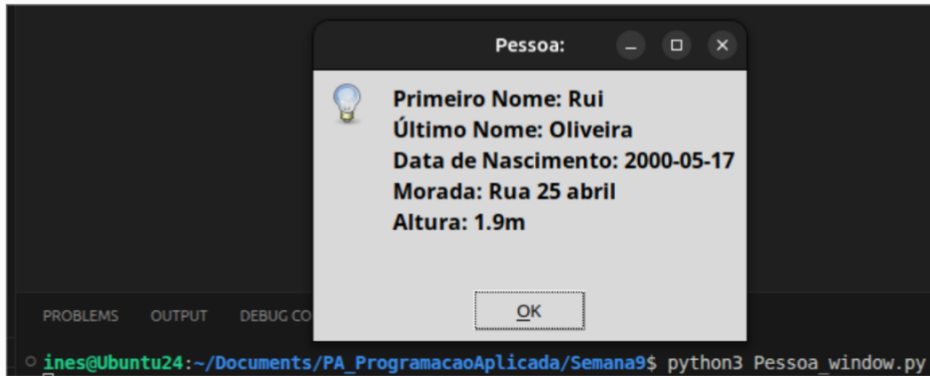
```
ines@Ubuntu24:~/Documents/PA_ProgramacaoAplicada/Semana9$ python3 ex1.py
Primeiro Nome: Rui
Último Nome: Oliveira
Data de Nascimento: 2000-05-17
Morada: Rua 25 abril
Altura: 1.9m
```

2.

Código com a função que permite apresentar os vários campos numa janela TKinter (usando um **pop-up** apenas contendo a informação da Pessoa):

```
Semana9 > Pessoa_window.py > ...
1  #from datetime import datetime
2  from datetime import date
3  from ex1 import Pessoa
4  import tkinter.messagebox
5
6  def show_person(person : Pessoa):
7      #top = tkinter.Tk()
8      #top.geometry("800x600")
9
10     mostra_pessoa = tkinter.messagebox.showinfo("Pessoa:", person)
11
12     mostra_pessoa.pack()
13     #top.mainloop()
14
15
16  #código de teste do Pessoa_window.py:
17  if __name__ == "__main__":
18
19      pessoa_1 = Pessoa("Rui", "Oliveira", date(2000, 5, 17), "Rua 25 abril", 1.90)
20      show_person(pessoa_1)
```

Testando a Message Box criada com os atributos da pessoa_1:



OU

Código com a classe LabelAtributo, subclasse de uma Frame TKinter, que irá inicializar o parent (uma janela ou Frame), e conter duas Labels (a primeira com o título do campo da pessoa - neste caso -, e a segunda label com o conteúdo desse mesmo campo):

```
Semana9 > Pessoa_window.py > show_person
1  #from datetime import datetime
2  from datetime import date
3  from ex1 import Pessoa
4  import tkinter as tk
5
6  class LabelAtributo(tk.Frame):
7      def __init__(self, parent, title, text):
8          super().__init__(parent)
9          self.lable_title = tk.Label(self, text=title)
10         self.lable_text = tk.Label(self, text=text)
11
12         self.lable_title.pack(side=tk.LEFT) #Alinhado à esquerda do anterior (parent)
13         self.lable_text.pack(side=tk.LEFT) #Alinhado à esquerda do anterior (entry)
14
```

A seguinte função **show_person** vai criar um objeto do tipo Janela Gráfica, mudando o seu tamanho, e com título "Pessoa". De seguida, criam-se tantos objetos do tipo LabelAtributo quantos os campos da Pessoa (que é passada como argumento da função), em que em cada LabelAtributo se passa a janela gráfica como o 'parent' frame, uma String que vai ser o 'título' desse campo da Pessoa, e o conteúdo desse campo/atributo da Pessoa. Por fim, faz-se o pack a cada LabelAtributo criado para os renderizar pela ordem desejada, e o mainloop do objeto Janela Gráfica, que mantém a janela ativa até se clicar na cruz depois de executada:

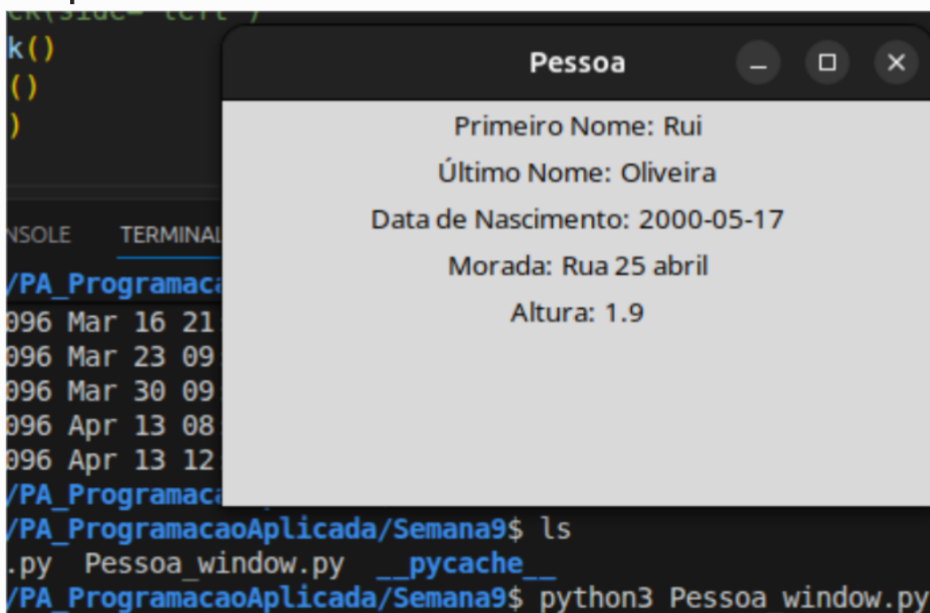

```

Semana9 > Pessoa_window.py > show_person
15
16 def show_person(person : Pessoa):
17     mostra_pessoa = tk.Tk() # cria objeto do tipo Janela gráfica
18     mostra_pessoa.geometry("350x200")
19     mostra_pessoa.title("Pessoa")
20
21
22     # Para aparecer Pop-up:
23     #tk.messagebox.showinfo("Pessoa:", person)
24
25     first_name = LabelAtributo(mostra_pessoa,"Primeiro Nome:", person.first_name)
26     last_name = LabelAtributo(mostra_pessoa,"Último Nome:", person.last_name )
27     birthday = LabelAtributo(mostra_pessoa,"Data de Nascimento:", person.birthdate)
28     address = LabelAtributo(mostra_pessoa,"Morada:", person.address)
29     height = LabelAtributo(mostra_pessoa,"Altura:", person.height)
30
31
32     # first_name = tk.Label(mostra_pessoa, text=p)
33     #first_name_title = tk.Label(mostra_pessoa, text=)
34     # last_name = tk.Label(mostra_pessoa, text=person.last_name)
35     # birthday = tk.Label(mostra_pessoa, text=person.birthdate)
36     # address = tk.Label(mostra_pessoa, text=person.address)
37     # height = tk.Label(mostra_pessoa, text=person.height)
38
39     # Criar botão:
40     #b = tk.Button(mostra_pessoa, text="Pessoa")
41     #b.pack() # o pack renderiza por ordem
42     #first_name_title.pack(side='left')
43     #first_name.pack(side='left')
44     first_name.pack()
45     last_name.pack()
46     birthday.pack()
47     address.pack()
48     height.pack()
49
50     mostra_pessoa.mainloop() # ciclo infinito da janela (programa, depois de executado, só acaba quando eu clicar na cruz para fechar)
51
52
53 #código de teste do Pessoa_window.py:
54 if __name__ == "__main__":
55
56     pessoa_1 = Pessoa("Rui", "Oliveira", date(2000, 5, 17), "Rua 25 abril", 1.90)
57     show_person(pessoa_1)

```

Para testar, cria-se uma pessoa com todos os atributos necessários nos formatos corretos, e de seguida chama-se a função `show_person` criada, passando a pessoa criada como argumento.

Por fim, testando no terminal, conseguimos observar a janela com todos os campos da pessoa criada, com os devidos títulos dos campos à sua esquerda:



3.

Semana9 > Modos.py > LabelEntry > entra_modos_edicao

```
1  #from datetime import datetime
2  from datetime import date
3  from ex1 import Pessoa
4  import tkinter as tk
5  import json
6
7  class LabelEntry(tk.Frame):
8      def __init__(self, parent, title, text):
9          super().__init__(parent)
10
11         self.textvar = tk.StringVar()
12         self.textvar.set(text)
13
14         self.edit = False # Modo Não-Edição
15
16         self.lable_title = tk.Label(self, text=title)
17         self.lable_text = tk.Label(self, text=text)
18         self.entry = tk.Entry(self, textvariable=self.textvar)
19
20         self.lable_title.pack(side=tk.LEFT) #Alinhado à esquerda do anterior (parent)
21         self.lable_text.pack(side=tk.LEFT) #Alinhado à esquerda do anterior (entry)
22
```

Semana9 > Modos.py > ...

```
7  class LabelEntry(tk.Frame):
22
23      def entra_modos_edicao(self):
24          if self.edit:
25              return
26          self.edit = True # Modo Edição
27
28          self.lable_text.pack_forget()
29          self.entry.pack(side=tk.LEFT)
30
31          # criar outra função dentro da classe -> sair_modos_edicao(self) :
32      def sair_modos_edicao(self):
33          if not self.edit:
34              return
35          self.edit = False
36
37          #Atualiza o texto do label com o que está no Entry
38          self.lable_text.config(text=self.textvar.get())
39
40          self.entry.pack_forget() # faz o unpack da Entry
41          self.lable_text.pack(side=tk.LEFT) # faz o pack do texto da entry para uma label, ao trocar de modo
42
```

```

43
44
45 def show_person(person : Pessoa):
46     mostra_pessoa = tk.Tk()      # cria objeto do tipo Janela gráfica
47     mostra_pessoa.geometry("350x200")
48     mostra_pessoa.title("Pessoa")
49
50
51     # Para aparecer Pop-up:
52     #tk.messagebox.showinfo("Pessoa:", person)
53
54     # widgets criados:
55     first_name = LabelEntry(mostra_pessoa,"Primeiro Nome:", person.first_name)
56     last_name = LabelEntry(mostra_pessoa,"Último Nome:", person.last_name )
57     birthday = LabelEntry(mostra_pessoa,"Data de Nascimento:", person.birthdate)
58     address = LabelEntry(mostra_pessoa,"Morada:", person.address)
59     height = LabelEntry(mostra_pessoa,"Altura:", person.height)
60
61
62     # entrar em modo Edição:
63     def editar():
64         first_name.entra_modos_edicao()
65         last_name.entra_modos_edicao()
66         birthday.entra_modos_edicao()
67         address.entra_modos_edicao()
68         height.entra_modos_edicao()
69
70     # criar função visualizar() para entrar no modo de Visualização:
71     def visualizar():
72         first_name.sair_modos_edicao()
73         last_name.sair_modos_edicao()
74         birthday.sair_modos_edicao()
75         address.sair_modos_edicao()
76         height.sair_modos_edicao()
77

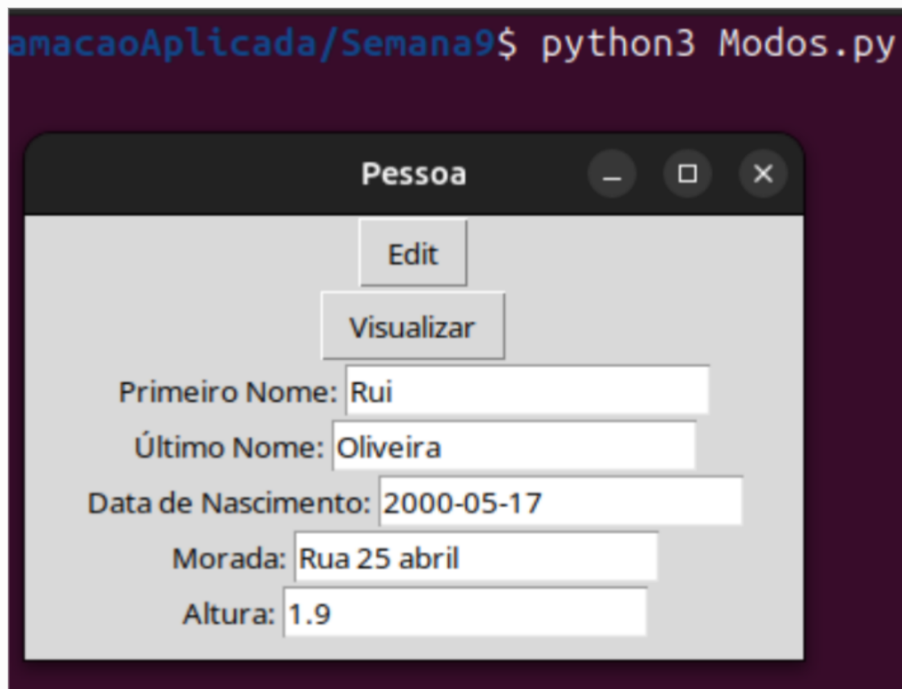
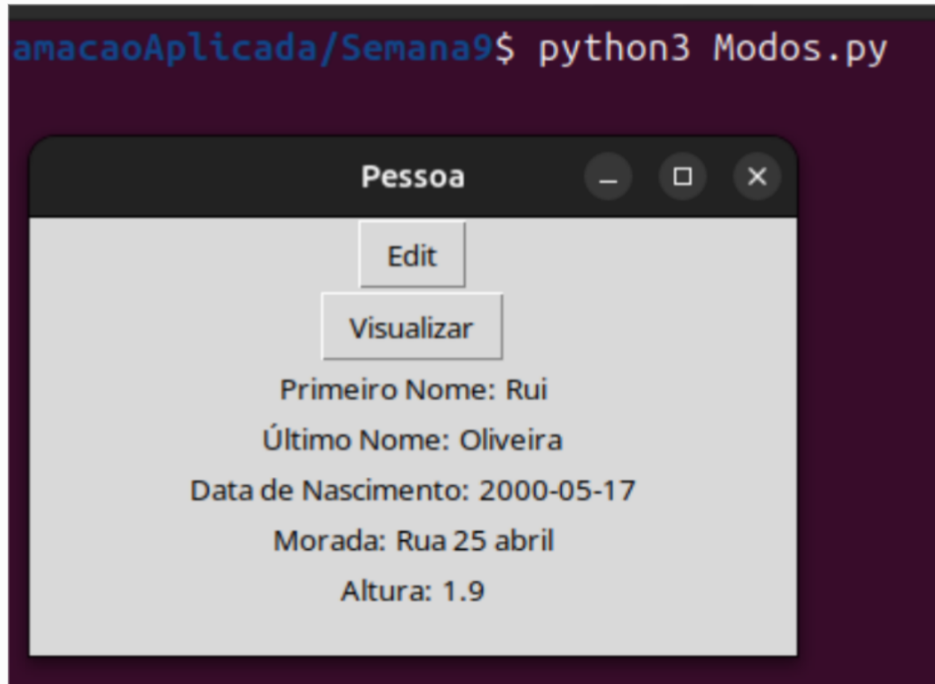
```

```

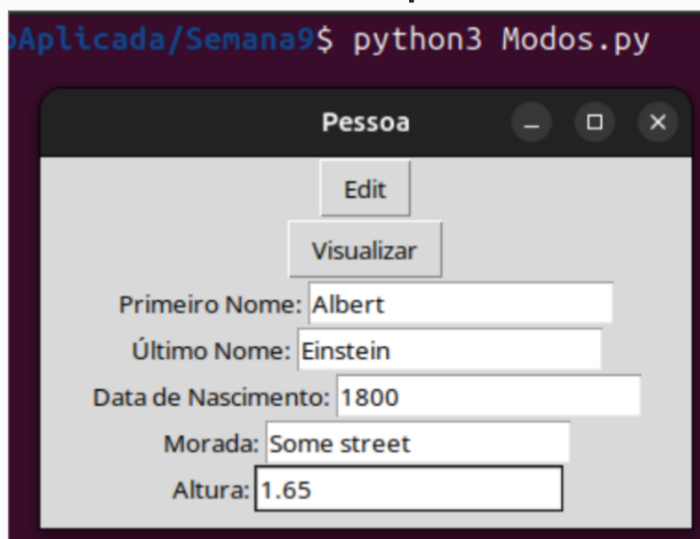
45 def show_person(person : Pessoa):
77
78
79     # Criar botão:
80     b_edit = tk.Button(mostra_pessoa, text="Edit", command=editar)
81
82     # Outro botão (para retomar ao modo de Visualização):
83     b_visualize = tk.Button(mostra_pessoa, text="Visualizar", command=visualizar)
84
85     b_edit.pack()                # o pack renderiza por ordem
86     b_visualize.pack()
87
88
89     first_name.pack()
90     last_name.pack()
91     birthday.pack()
92     address.pack()
93     height.pack()
94
95     mostra_pessoa.mainloop()    # ciclo infinito da janela (programa, depois de executado, só acaba quando eu clicar na cruz)
96
97
98 #código de teste do Modos.py:
99 if __name__ == "__main__":
100
101     pessoa_1 = Pessoa("Rui", "Oliveira", date(2000, 5, 17), "Rua 25 abril", 1.90)
102     show_person(pessoa_1)

```

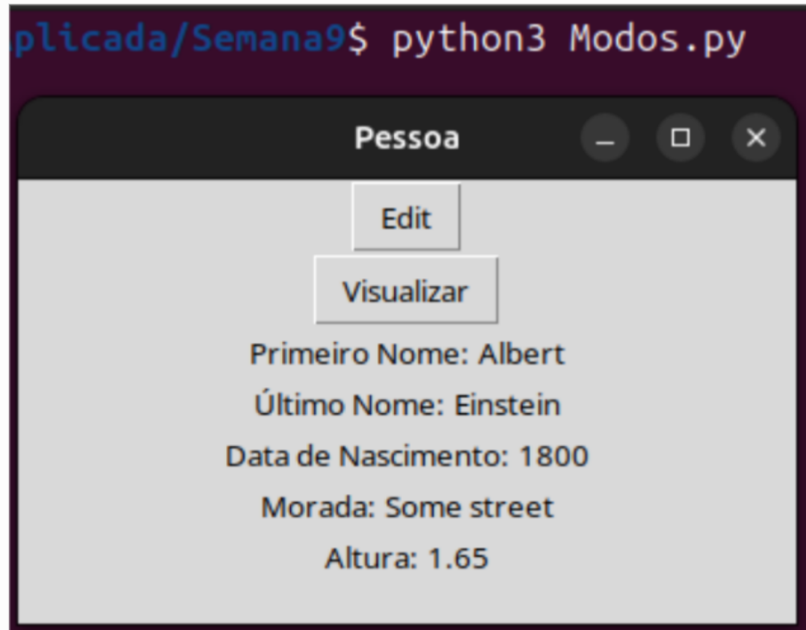
Ao executar no terminal:



Editando os campos:



Voltando ao Modo Visualização:



No entanto, não consegui implementar corretamente a auto-validação dos widgets a tempo.